What is Flutter?

Flutter is an **open-source UI (User Interface) software development kit (SDK)** created by **Google**. It is used to build **natively compiled applications** for **mobile (Android & iOS), web, desktop (Windows, macOS, Linux), and embedded devices** from a **single codebase**.

Flutter uses the **Dart programming language** and provides a rich set of **pre-designed widgets** that help developers create fast, beautiful, and responsive applications.

Why Should We Use Flutter?

Flutter is popular among students and developers because of the following reasons:

1. Single Codebase

- Write **one code** and run it on **multiple platforms** (Android, iOS, Web, Desktop).
- Saves development time and cost.

2. Fast Development (Hot Reload)

- Flutter supports **Hot Reload**, which allows developers to see changes instantly without restarting the app.
- Very helpful for learning and debugging.

3. High Performance

- Flutter apps are **compiled directly to native ARM code**.
- Performance is close to native Android and iOS apps.

4. Beautiful UI

- Flutter provides **customizable widgets**.
- Smooth animations and modern UI designs are easy to build.

5. Open Source & Backed by Google

- Free to use.
- Regular updates and strong community support.

6. Good Career Scope

- Used by companies like **Google, Alibaba, BMW, eBay**.
- High demand for Flutter developers.

History of Flutter

1. Who Invented Flutter?

- Flutter was developed by **Google**.

- The initial idea came from **Eric Seidel**, a former Google engineer.

2. Why Was Flutter Created?

Google wanted a framework that:

- Can run on multiple platforms
- Gives high performance
- Allows custom UI designs
- Reduces dependency on platform-specific UI components

3. When Was Flutter Introduced?

- **2015**: Flutter was first introduced as an experimental project called **Sky**.
- **2017**: Flutter was officially announced in **Alpha version**.
- **December 2018**: Flutter **1.0** was released.

Dart Language (Used in Flutter)

Flutter uses **Dart**, which is also developed by Google.

Features of Dart:

- Easy to learn (similar to C, Java, JavaScript)
- Object-Oriented
- Supports async/await
- Fast execution

Important Flutter Versions Over Time

Flutter Version Timeline

Year	Version	Key Features
2015	Sky	Early experimental version
2017	Alpha	Initial public preview
2018	Flutter 1.0	Stable release for Android & iOS
2019	Flutter 1.12	Web support (beta), desktop preview
2020	Flutter 1.17	Improved performance & UI tools
2021	Flutter 2.0	Stable support for Web & Desktop
2022	Flutter 3.0	macOS & Linux stable support
2023	Flutter 3.x	Performance improvements, Material 3
2024–2025	Flutter 3.x+	AI integration, better tooling & stability

Platforms Supported by Flutter

- Android, iOS, Web, Windows, macOS, Linux, Embedded systems

Where Flutter is Used?

- Mobile Applications
- Web Applications
- Desktop Software
- Startup Apps
- Enterprise Applications

Advantages of Flutter

- One codebase
- Fast UI development
- Near-native performance
- Strong community
- Free & open-source

Disadvantages of Flutter

- App size is larger
- Limited libraries compared to native (but improving)
- Dart language is less popular than JavaScript

Conclusion

Flutter is a powerful framework for students and developers who want to build **modern, fast, and cross-platform applications**. With Google's support and growing demand in the industry, Flutter is an excellent choice for a career in **app development**.

- Flutter is developed by **Google**
- Uses **Dart language**
- First stable version: **Flutter 1.0 (2018)**
- Supports **Android, iOS, Web & Desktop**

Complete Flutter Installation & Setup Guide

System Requirements

- OS: Windows 10+, macOS, or Linux
- RAM: Minimum 8 GB (Recommended)
- Disk Space: At least 10 GB free
- Internet connection

Install Flutter SDK

Step 1: Download Flutter

1. Open a web browser
2. Visit the **official Flutter website**
3. Download Flutter SDK according to your OS (Windows / macOS / Linux)

Step 2: Extract Flutter SDK

- Extract the downloaded ZIP file
- Place it in a fixed location:

Windows

```
C:\flutter
```

macOS / Linux

```
~/development/flutter
```

Set Up Environment Variables (PATH)

For Windows

1. Open **This PC → Properties**
2. Click **Advanced system settings**
3. Click **Environment Variables**
4. Under **System Variables**, select `Path`
5. Click **Edit → New**
6. Add:

```
C:\flutter\bin
```

7. Click OK and restart the system

Verify Flutter Installation

Open **Command Prompt / Terminal** and run:

```
flutter doctor
```

- This command checks all required dependencies
- Green check means everything is correctly installed

5. Install Android Studio

Step 1: Download & Install

- Download Android Studio from the official website
- Install with **default settings**

Step 2: Required Components

Make sure these are installed:

- Android SDK
- Android SDK Platform-Tools
- Android SDK Build-Tools

- Android Emulator

6. Configure Android SDK Path

1. Open **Android Studio**
2. Go to **Settings → Appearance & Behavior → System Settings → Android SDK**
3. Note SDK path (example):

```
C:\Users\YourName\AppData\Local\Android\Sdk
```

Flutter usually detects it automatically.

Install Flutter & Dart Plugins

1. Open Android Studio
2. Go to **Settings → Plugins**
3. Install:
    - Flutter
    - Dart (auto-installed)
4. Restart Android Studio

Set Up Android Emulator

1. Open Android Studio
2. Go to **Device Manager**
3. Click **Create Device**
4. Select Pixel device
5. Download system image
6. Start the emulator

Create a New Flutter Project (Android Studio)

Steps:

1. Open Android Studio
2. Click **New Flutter Project**
3. Select **Flutter Application**
4. Set Flutter SDK path:

```
C:\flutter
```

5. Enter details:
    - Project Name: `my_first_flutter_app`
    - Language: Dart
6. Click **Finish**

Flutter creates a default project automatically.

Run Your First Flutter App

Using Emulator

- Start emulator
- Click **Run** button

Using Terminal

```
flutter run
```

Flutter Project Structure

- `lib/` → Main code (`main.dart`)
- `android/` → Android specific files
- `ios/` → iOS specific files
- `pubspec.yaml` → Dependencies & assets

Common Errors & Solutions

Flutter Doctor Issues

- Follow suggested commands
- Install missing SDKs

Emulator Not Detected

- Enable virtualization in BIOS
- Install Intel HAXM / Hypervisor

Flutter Widgets Explained (With Diagrams)

1. What is a Widget in Flutter?

In Flutter, **everything is a widget**.

- Buttons, text, images, layout, padding – all are widgets.
- Widgets describe **how the UI should look**.

Think of widgets like **building blocks** of an app UI.

2. Types of Widgets in Flutter

Flutter widgets are mainly divided into **three categories**:

(A) Stateless Widget

- UI does **not change** once built
- No internal state

Examples: Text, Icon, Row, Column

Diagram (Conceptual):

[StatelessWidget]
|
UI Fixed

(B) Stateful Widget

- UI **can change** during runtime
- Uses `setState()`

Examples: Checkbox, Counter App, Switch

Diagram (Conceptual):

[StatefulWidget]
|
State Changes
|
UI Updates

(C) Layout Widgets

Used to arrange widgets on the screen.

Examples: Row, Column, Container, Stack

Diagram (Row Example):

[Row]
| | |
W1 W2 W3

3. Widget Tree Concept

Flutter UI is built as a **Widget Tree**.

Example Diagram:

MaterialApp
└── Scaffold
├── AppBar
└── Body
└── Center
└── Text

First Flutter App – Counter App

This is the **default Flutter Counter App** created when you start a new project.

1. Full Counter App Code (main.dart)

```
import 'package:flutter/material.dart';
```

```
void main() {
  runApp(const MyApp());
}
```

……

2. Line-by-Line Explanation

`import 'package:flutter/material.dart';`

- Imports Material Design widgets

`void main()`

- Entry point of Flutter app

`runApp(MyApp())`

- Runs the root widget

`StatelessWidget (MyApp)`

- App-level widget
- UI does not change

`MaterialApp`

- Provides theme, navigation, title

`StatefulWidget (MyHomePage)`

- Used because counter value changes

`_counter` variable

- Stores counter value

`setState()`

- Tells Flutter to rebuild UI
- Updates screen when data changes

`Scaffold`

- Provides basic app layout
- AppBar, Body, FloatingActionButton

`FloatingActionButton`

- Button to increase counter

Counter App Flow Diagram

User taps + button
↓
_incrementCounter()
↓
setState()
↓
UI Rebuilds
↓
Counter Value Updated

Learning Summary

- Widgets are building blocks
- Stateless = fixed UI
- Stateful = dynamic UI
- `setState()` updates UI
- Counter App explains Flutter basics clearly

Step-by-Step Guide For Create AVD

These notes explain **how to create an Android Virtual Device (AVD)**, **run a Flutter application on emulator, real Android phone, desktop, and Chrome browser**, and **how to enable Developer Options and connect a phone for Flutter development**.

Prerequisites (Must Install First)

Before starting Flutter development, make sure the following are installed:

1. **Flutter SDK**
2. **Android Studio**
3. **Android SDK & Platform Tools** (comes with Android Studio)
4. **VS Code** (recommended) or Android Studio IDE
5. **Google Chrome** (for web apps)

After installation, open **Command Prompt / Terminal** and run:

```
flutter doctor
```

This command checks whether everything is installed correctly.

How to Create Android Virtual Device (AVD)

Step 1: Open Android Studio

- Launch **Android Studio**
- Click on **More Actions → Virtual Device Manager**

Step 2: Create a New Virtual Device

- Click **Create Device**
- Choose a device (example: **Pixel 4 / Pixel 6**)

- Click **Next**

Step 3: Select Android Version

- Choose a system image (recommended: **Android 12 / 13 – x86_64**)
- If not downloaded, click **Download**
- Click **Next → Finish**

Step 4: Start Emulator

- Click the **Play button** in Virtual Device Manager
- Emulator will start like a real Android phone

Run Flutter App on Android Emulator

Step 1: Open Flutter Project

- Open project in **VS Code** or **Android Studio**

Step 2: Check Connected Devices

```
flutter devices
```

You should see your emulator listed.

Step 3: Run App

```
flutter run
```

App will launch on the Android Emulator

Enable Developer Options on Android Phone

Step 1: Open Phone Settings

- Go to **Settings → About Phone**

Step 2: Enable Developer Mode

- Tap **Build Number 7 times**
- You will see: *You are now a developer*

Step 3: Enable USB Debugging

- Go to **Settings → Developer Options**
- Turn ON **USB Debugging**

Connect Android Phone for Flutter Development (USB)

Step 1: Connect Phone to PC

- Use USB cable
- Select **File Transfer (MTP)** mode

Step 2: Allow USB Debugging

- A popup appears on phone
- Click **Allow**

Step 3: Verify Connection

```
flutter devices
```

Your phone name should appear

Step 4: Run App

```
flutter run
```

App runs on real Android device

Connect Android Phone Using Same WiFi (Wireless Debugging)

Step 1: Enable Wireless Debugging

- Settings → Developer Options
- Turn ON **Wireless Debugging**

Step 2: Connect via USB First

```
adb devices
```

Step 3: Enable TCP/IP Mode

```
adb tcpip 5555
```

Step 4: Get Phone IP Address

- Settings → About Phone → Status → IP Address

Step 5: Connect Wirelessly

```
adb connect <phone_ip>:5555
```

Example:

```
adb connect 192.168.1.5:5555
```

Step 6: Run Flutter App

```
flutter run
```

Phone must be connected to **same WiFi** as PC

Run Flutter App on Desktop (Windows)

Step 1: Enable Windows Desktop Support

```
flutter config --enable-windows-desktop
```

Step 2: Check Devices

```
flutter devices
```

You will see **Windows** listed

Step 3: Run App on Desktop

```
flutter run -d windows
```

Flutter app runs as Windows desktop application

Run Flutter App on Chrome (Web)

Step 1: Enable Web Support

```
flutter config --enable-web
```

Step 2: Check Devices

```
flutter devices
```

You should see **Chrome** listed

Step 3: Run App on Chrome

```
flutter run -d chrome
```

App opens in Chrome browser

Common Problems & Solutions

Device not detected

- Check USB Debugging
- Run `flutter doctor`
- Restart adb:

```
adb kill-server
adb start-server
```

Emulator slow

- Enable **Virtualization** in BIOS
- Use **x86_64 system image**

Summary (For Students)

- Emulator → Practice without phone
- Real device → Best performance testing
- Same WiFi → Wireless testing
- Desktop & Chrome → Multi-platform apps

Flutter allows **One Codebase → Multiple Platforms** 🚀

These notes are suitable for **classroom teaching, PDFs, and student handouts**

Running a Flutter App on Desktop

(Windows, macOS, Linux)

Flutter supports **desktop applications**, but **only on the operating system you are using**.

Desktop Platforms Supported by Flutter

Platform	Can Build On
Windows Desktop	Windows OS
macOS Desktop	macOS only
Linux Desktop	Linux OS
iOS (iPhone/iPad)	macOS only
Android	Windows / macOS / Linux

How to Run a Flutter App on Windows Desktop

Step 1: Install Required Software

Make sure these are installed:

1. **Flutter SDK**
2. **Git**
3. **Visual Studio 2022**
    - Select:
        - Desktop development with C++
        - Windows 10/11 SDK

Step 2: Enable Windows Desktop Support

Open **Command Prompt** and run:

```
flutter config --enable-windows-desktop
```

Step 3: Check Setup

Run:

```
flutter doctor
```

You should see:

```
[✓] Windows desktop device available
```

Step 4: Create or Open a Flutter Project

```
flutter create my_desktop_app
cd my_desktop_app
```

Step 5: Run App on Windows Desktop

```
flutter run -d windows
```

A **Windows desktop window** will open with your Flutter app.

3. How Flutter Desktop App Works (Simple Explanation)

- Flutter uses **native system APIs**
- Windows apps need:
  - Windows SDK
  - MSVC compiler
  - Windows system libraries
- Flutter automatically converts Dart code → native Windows app

4. Why We Cannot Run macOS Desktop App on Windows

Main Reasons:

1. **macOS apps require Apple frameworks**
   - Cocoa
   - AppKit
2. These frameworks exist **only on macOS**
3. Apple **does not allow** macOS app building on Windows
4. Xcode is required → **Xcode runs only on macOS**

Windows cannot compile macOS apps
Apple tools are not available for Windows

5. Why We Cannot Run iOS Apps on Windows

iOS App Requirements:

Requirement	Availability
Xcode	macOS only
iOS Simulator	macOS only
Apple SDK	macOS only
Code Signing	Apple Developer Account

Apple **restricts iOS development to macOS**

That is why:

```
Windows → iOS App
macOS → iOS App
```

Important Concept for Students (Exam / Interview Point)

You can build apps only for the OS you are currently using.

Your OS	You Can Build
Windows	Windows, Android, Web
macOS	macOS, iOS, Android, Web
Linux	Linux, Android, Web

Can We Use Any Trick to Run iOS on Windows?

Not Recommended for Students

- Virtual macOS → illegal (Apple license violation)
- Cloud Mac services → paid & limited
- Hackintosh → unstable & unsafe

Best practice: Use a **Mac system** for iOS/macOS apps

Summary (Easy to Remember)

- Flutter supports **desktop apps**
- Windows → Windows app only
- macOS → macOS + iOS apps
- iOS apps need **Xcode + macOS**
- Apple does **not allow iOS/macOS builds on Windows**

Step-by-Step Guide For Create AVD

These notes explain **how to create an Android Virtual Device (AVD)**, **run a Flutter application on emulator, real Android phone, desktop, and Chrome browser**, and **how to enable Developer Options and connect a phone for Flutter development**.

Prerequisites (Must Install First)

Before starting Flutter development, make sure the following are installed:

6. **Flutter SDK**
7. **Android Studio**
8. **Android SDK & Platform Tools** (comes with Android Studio)
9. **VS Code** (recommended) or Android Studio IDE
10. **Google Chrome** (for web apps)

After installation, open **Command Prompt / Terminal** and run:

```
flutter doctor
```

This command checks whether everything is installed correctly.

How to Create Android Virtual Device (AVD)

Step 1: Open Android Studio

- Launch **Android Studio**
- Click on **More Actions → Virtual Device Manager**

Step 2: Create a New Virtual Device

- Click **Create Device**
- Choose a device (example: **Pixel 4 / Pixel 6**)
- Click **Next**

Step 3: Select Android Version

- Choose a system image (recommended: **Android 12 / 13 – x86_64**)
- If not downloaded, click **Download**
- Click **Next → Finish**

Step 4: Start Emulator

- Click the **Play button** in Virtual Device Manager
- Emulator will start like a real Android phone

Run Flutter App on Android Emulator

Step 1: Open Flutter Project

- Open project in **VS Code** or **Android Studio**

Step 2: Check Connected Devices

```
flutter devices
```

You should see your emulator listed.

Step 3: Run App

```
flutter run
```

App will launch on the Android Emulator

Enable Developer Options on Android Phone

Step 1: Open Phone Settings

- Go to **Settings → About Phone**

Step 2: Enable Developer Mode

- Tap **Build Number 7 times**
- You will see: *You are now a developer*

Step 3: Enable USB Debugging

- Go to **Settings → Developer Options**
- Turn ON **USB Debugging**

Connect Android Phone for Flutter Development (USB)

Step 1: Connect Phone to PC

- Use USB cable
- Select **File Transfer (MTP)** mode

Step 2: Allow USB Debugging

- A popup appears on phone
- Click **Allow**

Step 3: Verify Connection

```
flutter devices
```

Your phone name should appear

Step 4: Run App

```
flutter run
```

App runs on real Android device

Connect Android Phone Using Same WiFi (Wireless Debugging)

Step 1: Enable Wireless Debugging

- Settings → Developer Options
- Turn ON **Wireless Debugging**

Step 2: Connect via USB First

```
adb devices
```

Step 3: Enable TCP/IP Mode

```
adb tcpip 5555
```

Step 4: Get Phone IP Address

- Settings → About Phone → Status → IP Address

Step 5: Connect Wirelessly

```
adb connect <phone_ip>:5555
```

Example:

```
adb connect 192.168.1.5:5555
```

Step 6: Run Flutter App

```
flutter run
```

Phone must be connected to **same WiFi** as PC

Run Flutter App on Desktop (Windows)

Step 1: Enable Windows Desktop Support

```
flutter config --enable-windows-desktop
```

Step 2: Check Devices

```
flutter devices
```

You will see **Windows** listed

Step 3: Run App on Desktop

```
flutter run -d windows
```

Flutter app runs as Windows desktop application

Run Flutter App on Chrome (Web)

Step 1: Enable Web Support

```
flutter config --enable-web
```

Step 2: Check Devices

```
flutter devices
```

You should see **Chrome** listed

Step 3: Run App on Chrome

```
flutter run -d chrome
```

App opens in Chrome browser

Common Problems & Solutions

Device not detected

- Check USB Debugging
- Run `flutter doctor`
- Restart adb:

```
adb kill-server
adb start-server
```

Emulator slow

- Enable **Virtualization** in BIOS
- Use **x86_64 system image**

Summary (For Students)

- Emulator → Practice without phone
- Real device → Best performance testing
- Same WiFi → Wireless testing
- Desktop & Chrome → Multi-platform apps

Flutter allows **One Codebase → Multiple Platforms** 🚀

These notes are suitable for **classroom teaching, PDFs, and student handouts**

Running a Flutter App on Desktop

(Windows, macOS, Linux)

Flutter supports **desktop applications**, but **only on the operating system you are using**.

Desktop Platforms Supported by Flutter

Platform	Can Build On
Windows Desktop	Windows OS
macOS Desktop	macOS only
Linux Desktop	Linux OS
iOS (iPhone/iPad)	macOS only
Android	Windows / macOS / Linux

How to Run a Flutter App on Windows Desktop

Step 1: Install Required Software

Make sure these are installed:

4. **Flutter SDK**

5. **Git**
6. **Visual Studio 2022**
    - Select:
        - Desktop development with C++
        - Windows 10/11 SDK

Step 2: Enable Windows Desktop Support

Open **Command Prompt** and run:

```
flutter config --enable-windows-desktop
```

Step 3: Check Setup

Run:

```
flutter doctor
```

You should see:

```
[✓] Windows desktop device available
```

Step 4: Create or Open a Flutter Project

```
flutter create my_desktop_app
cd my_desktop_app
```

Step 5: Run App on Windows Desktop

```
flutter run -d windows
```

A **Windows desktop window** will open with your Flutter app.

3. How Flutter Desktop App Works (Simple Explanation)

- Flutter uses **native system APIs**
- Windows apps need:
    - Windows SDK
    - MSVC compiler
    - Windows system libraries
- Flutter automatically converts Dart code → native Windows app

4. Why We Cannot Run macOS Desktop App on Windows

Main Reasons:

5. **macOS apps require Apple frameworks**
    - Cocoa
    - AppKit
6. These frameworks exist **only on macOS**
7. Apple **does not allow** macOS app building on Windows
8. Xcode is required → **Xcode runs only on macOS**

Windows cannot compile macOS apps
Apple tools are not available for Windows

5. Why We Cannot Run iOS Apps on Windows

iOS App Requirements:

Requirement	Availability
Xcode	macOS only
iOS Simulator	macOS only
Apple SDK	macOS only
Code Signing	Apple Developer Account

Apple **restricts iOS development to macOS**

That is why:

```
Windows → iOS App
macOS → iOS App
```

Important Concept for Students (Exam / Interview Point)

You can build apps only for the OS you are currently using.

Your OS	You Can Build
Windows	Windows, Android, Web
macOS	macOS, iOS, Android, Web
Linux	Linux, Android, Web

Can We Use Any Trick to Run iOS on Windows?

Not Recommended for Students

- Virtual macOS → illegal (Apple license violation)
- Cloud Mac services → paid & limited
- Hackintosh → unstable & unsafe

Best practice: Use a **Mac system** for iOS/macOS apps

Summary (Easy to Remember)

- Flutter supports **desktop apps**
- Windows → Windows app only
- macOS → macOS + iOS apps
- iOS apps need **Xcode + macOS**
- Apple does **not allow iOS/macOS builds on Windows**